Bug Report

Nexo InOuty backend (API): defects found while adding automated tests

29 October 2026 · branch `test/coverage`

29	1	10	18
bugs in total	High severity	Medium severity	Low severity

About this report

Every bug below was found while writing automated tests for the backend API (99.8% of lines and 93.7% of branches covered), run against the real application and a real MongoDB database. Each one is **proven by a test** that describes the correct behaviour and currently fails. The test is marked `it.failing` and named in the "Proven by" line. When a bug is fixed, its test starts passing and the test runner flags it, so the marker can be removed.

Severity. High: one company can reach another company's data, or wrong pay / lost data on a main workflow for every company. **Medium**: wrong pay, lost data or a blocked workflow in specific but common situations (night shifts, a second tab, copying a month). **Low**: wrong messages, time-zone display, crashes on requests the app never sends, rare races.

Effect says what goes wrong for the business or the user. **What happens** is the observed behaviour in the test. Some bugs are also guarded by the frontend today; they are listed because another client, an old browser tab or a direct API call still reaches them.

Bugs by area

Area	High	Medium	Low	Total
Common, tenancy & activity log	–	–	5	**5**
Sign-in, users & companies	–	–	4	**4**
Employees, shifts & tasks	–	2	3	**5**
Planning	–	3	4	**7**
Work time, advances & bonuses	–	2	1	**3**
Attendance, kiosk & machines	1	3	1	**5**

All bugs at a glance

ID	Severity	Area	Bug
BUG-501	**High**	Attendance, kiosk & machines	Every company reads the same clocking machine, so one company gets another's punches
BUG-203	**Medium**	Employees, shifts & tasks	A manager can raise their own salary
BUG-204	**Medium**	Employees, shifts & tasks	A shift still used in the planning is deleted at once
BUG-303	**Medium**	Planning	Importing a day crashes when one of its employees or shifts was deleted
BUG-304	**Medium**	Planning	Copying a month deletes the whole destination month, also the weekdays it doesn't copy
BUG-306	**Medium**	Planning	The exports leave out shifts starting at 13:00, before 06:00 and in the evening
BUG-401	**Medium**	Work time, advances & bonuses	An advance of 0 or a negative amount is accepted
BUG-403	**Medium**	Work time, advances & bonuses	Two work time records can be saved for the same employee, shift and day
BUG-502	**Medium**	Attendance, kiosk & machines	After the machine's log is cleared, its new punches are dropped without a word
BUG-504	**Medium**	Attendance, kiosk & machines	After a normal day, the kiosk thinks the employee is still clocked in the next morning
BUG-505	**Medium**	Attendance, kiosk & machines	Overtime after midnight is lost unless the shift ends at exactly 00:00
BUG-001	**Low**	Common, tenancy & activity log	A MongoDB operator nested more than 20 levels deep gets past the injection check
BUG-002	**Low**	Common, tenancy & activity log	Two error messages reach French and Arabic users in English
BUG-003	**Low**	Common, tenancy & activity log	A clock-out at exactly 00:00 is logged as "Clock-out time: Changed"
BUG-004	**Low**	Common, tenancy & activity log	The activity and error logs filter days in UTC, so in Algeria 00:00–00:59 lands on the day before
BUG-005	**Low**	Common, tenancy & activity log	A change the database refuses is logged as a 500, with Mongoose's raw text
BUG-101	**Low**	Sign-in, users & companies	Chrome, Firefox and Edge on phones are listed as another browser
BUG-102	**Low**	Sign-in, users & companies	Two admins demoting each other at the same moment leave the company with no admin
BUG-103	**Low**	Sign-in, users & companies	A second company whose name gives the same sign-in link name can't be created
BUG-104	**Low**	Sign-in, users & companies	A weekly advance share of 0% is refused, with a message saying 0 is allowed
BUG-201	**Low**	Employees, shifts & tasks	An edit can empty an employee's name
BUG-202	**Low**	Employees, shifts & tasks	An unknown id answers 200 with an empty body instead of 404
BUG-205	**Low**	Employees, shifts & tasks	The API accepts an employee without an employee number
BUG-301	**Low**	Planning	Copying a month without the months crashes with a 500
BUG-302	**Low**	Planning	Saving a day without the list of entries empties the day, then crashes
BUG-305	**Low**	Planning	"The first Tuesday" of a copied month is the Tuesday saved first, not the earliest one
BUG-307	**Low**	Planning	A sub-task without a start time makes the exports crash

ID	Severity	Area	Bug
BUG-402	**Low**	Work time, advances & bonuses	Advances and bonuses are accepted for an employee who doesn't exist
BUG-503	**Low**	Attendance, kiosk & machines	Clock-in History cuts days at UTC midnight, so in Algeria 00:00–00:59 lands on the day before

High severity · details

1 bug, sorted by ID.

BUG-501	Every company reads the same clocking machine, so one company gets another's punches	HIGH
Area	Attendance, kiosk & machines	
Effect	**Every company with the clock-in feature reaches the one clocking machine in the server settings. Another company's Sync copies your employees' punches into their data, and they can read the names on your machine. This is the only place where companies' data is not kept apart.**	
What happens	Every company talks to the one machine in `DEVICE_IP`. B's sync copies A's punches into B's attendance (where they match B's employee numbers, they count as B's employees' punches); `/device/users` returns the names on A's machine, `/device/logs` all A's punches, and `/device/status` the machine's IP. Everything else in the app is strictly separated by company; this is the one hole.	
Steps to reproduce	Company A owns the face-recognition machine at `DEVICE_IP`. An admin or manager of company B (any company with the clock-in feature) opens Clock-in History and presses Sync, or calls `GET /device/users` / `GET /device/logs`.	
Expected	Company B only reaches a machine registered to company B; with none, it gets nothing (or "no machine configured").	
Where	`src/attendance-device/attendance-device.service.ts`, `connect` (`new ZKLib(process.env.DEVICE_IP, …)`); used by `POST /attendance/sync` and `GET /device/{ping,status,users,logs}`	
Type	Security (data of another company)	
Suggested fix	Store each machine's address on the company (as `devices[].serial` already is for the ZKTeco push) and connect to the current company's machine; refuse when it has none. Until then, allow these routes only to the company that owns the machine.	
Proven by	`attendance.e2e-spec.ts` › "a company's sync never brings in punches from another company's machine", "a company can't read the user list and punches of another company's machine"	

Medium severity · details

10 bugs, sorted by ID.

BUG-203 A manager can raise their own salary — MEDIUM

Area	Employees, shifts & tasks
Effect	**A manager can raise their own salary through the API, and payroll pays it. They can also delete their own employee record.**
What happens	Saved. Payroll then pays the new salary. The same manager can also delete their own employee record.
Steps to reproduce	A manager whose account is linked to their employee record sends `PUT /employees/<own id>` with `{ "baseSalary": 99000 }`.
Expected	403. A manager can't see other managers' salaries, so changing their own should be for an admin only.
Where	`src/employees/employees.controller.ts`, `update` / `assertCanChange` (only *other* managers are protected)
Type	Security (permissions)
Suggested fix	In `assertCanChange`, also refuse the manager's own linked employee (`user.employeeId`), at least for `baseSalary`; or let managers change salaries only for employees who aren't managers.
Proven by	`staff.e2e-spec.ts` › "employees › as a manager › can't change their own salary"

BUG-204 A shift still used in the planning is deleted at once — MEDIUM

Area	Employees, shifts & tasks
Effect	**Deleting a shift that is still planned happens at once with no warning. Those planning entries point at nothing, show as an unknown shift and disappear from the exports.**
What happens	The shift is deleted straight away. The confirmation is never shown, and the planning entries keep a `shiftId` that points at nothing (they show as an unknown shift and drop out of the exports).
Steps to reproduce	Plan someone on the 08:00–16:00 shift, then delete that shift from Shifts management.
Expected	409 with `{ usageCount: 1 }`, as the frontend expects (its BUG-403 fix reads `usageCount` and asks "This shift is used N times, delete anyway?"); then `DELETE /shifts/:id?force=true` deletes it.
Where	`src/shifts/shifts.service.ts`, `remove(id, force)` (`force` is never used)
Type	Bug (data loss)
Suggested fix	Count planning entries with this `shiftId`; if any and `force` is false, throw `ConflictException({ message, usageCount })`. On force, decide what happens to those entries (delete them, or archive the shift instead).
Proven by	`staff.e2e-spec.ts` › "shifts › asks before deleting a shift that is still in the planning, then deletes with force"

BUG-303 — Importing a day crashes when one of its employees or shifts was deleted — MEDIUM

Area	Planning
Effect	**Once an employee or shift used on a day is deleted, that day can no longer be imported into another day: the import fails every time.**
What happens	500 "Something went wrong": `Cannot read properties of null (reading '_id')`. The whole day can no longer be imported.
Steps to reproduce	Plan Ali on a day, delete Ali (or delete the shift: BUG-204 makes that one easy), then import that day into another day in New planning.
Expected	The import works and leaves out the entries that point at nothing (as the exports already do).
Where	`src/planning/planning.service.ts`, `importFromDate` (`item.empId._id`, `item.shiftId._id`)
Type	Bug (crash)
Suggested fix	Skip entries whose populated `empId` or `shiftId` is null, like the export builders do.
Proven by	`planning.e2e-spec.ts` › "skips entries whose employee or shift was deleted"

BUG-304 — Copying a month deletes the whole destination month, also the weekdays it doesn't copy — MEDIUM

Area	Planning
Effect	**Copying one month's planning onto another deletes every day of the target month, including weekdays it doesn't copy. Planning already made for those days is lost without warning.**
What happens	Every day of November is deleted first, then only the Tuesdays are filled: the Thursday is gone. The copy dialog only says "Copy and save the planning from one month to another month."
Steps to reproduce	September has planning only on Tuesdays. November already has a Thursday planned. Copy September → November.
Expected	November's Tuesdays get September's Tuesday; its Thursday stays (or the user is told the whole month will be replaced).
Where	`src/planning/planning.service.ts`, `copyMonth` (`deleteMany` over the whole destination month)
Type	Data loss
Suggested fix	Delete only the destination days that receive a copy (the weekdays found in the source), or ask the frontend to warn that the month is replaced.
Proven by	`planning.e2e-spec.ts` › "keeps the destination days of weekdays the source month has no planning for"

BUG-306 — The exports leave out shifts starting at 13:00, before 06:00 and in the evening — MEDIUM

Area	Planning
Effect	**The planning exports (Excel and PDF) silently leave out everyone starting at 13:00, before 06:00 or in the evening; the manager week export has no evening service at all. Printed plannings are incomplete.**
What happens	Each export only has rows for a fixed list of hours, and anyone starting at another hour is silently dropped (`if (weekGrid[day][hour])`). None of the three has 13H or 22H–05H; the manager week export (with backups) stops at **16H**, so its whole evening service (17:00–21:00) is missing. The three lists were copied and changed separately.
Steps to reproduce	Plan someone on a shift starting at 13:00, 18:00, 22:00 or 00:00, then export the first week or the day (Excel or PDF).
Expected	Everyone planned that day is in the export, under their start hour.
Where	`src/planning/planning.service.ts`: the fixed `hours` lists in `buildFirstWeekPlanningWithBackup`, `buildFirstWeekPlanningWithoutBackups` and `buildDayPlanning`
Type	Bug (missing data)
Suggested fix	Build the hour rows from the start hours actually present (sorted), instead of a fixed list; share one grid builder between the three exports.
Proven by	`planning.e2e-spec.ts` › "… includes a shift starting at 13:00" (and 22:00, 18:00, 00:00)

BUG-401 — An advance of 0 or a negative amount is accepted — MEDIUM

Area	Work time, advances & bonuses
Effect	**A negative advance adds money to the payslip, and a 0 advance blocks the real advance for that date. Only the frontend prevents it today.**
What happens	Saved. A negative advance *adds* money to the payslip; a 0 advance blocks the real one for that date ("already exists"). The frontend fixed its side (its BUG-503), but any other client or an old tab can still send it.
Steps to reproduce	`POST /advances` with `{ emp_id: 4, amount: -5000, date: "2026-10-05" }`.
Expected	400, as bonuses do ("amount must be greater than 0").
Where	`src/advances/advances.service.ts`, `create` (no check on `amount`)
Type	Bug (money)
Suggested fix	Same check as bonuses: `Number.isFinite(amount) && amount > 0`, and refuse a date that isn't `YYYY-MM-DD`.
Proven by	`payroll.e2e-spec.ts` › "refuses an advance of 0" / "refuses an advance of -5000"

BUG-403 — Two work time records can be saved for the same employee, shift and day — MEDIUM

Area	Work time, advances & bonuses
Effect	**Two work time records can be saved for the same employee, shift and day (two tabs, the kiosk and a manager, a double click). Payroll counts both, so the day is paid twice.**
What happens	Two records. Payroll and the reports add both, so the day is paid twice. The frontend avoids it when it can (its BUG-015 / BUG-209 fixes) but the server doesn't guard it.
Steps to reproduce	Two saves for the same employee, shift and day arrive together (two tabs, the kiosk and a manager, or a double click): two `POST /worktime`.
Expected	One record (the second save updates it, or is refused).
Where	`src/worktime/worktime.service.ts` `create`; no unique index in `src/worktime/worktime.shema.ts`
Type	Bug (money)
Suggested fix	A unique index on `{ companyId, emp_id, shift_id, work_date }` and an upsert (`findOneAndUpdate(…, { upsert: true })`) in `create`. Clean existing duplicates first.
Proven by	`payroll.e2e-spec.ts` › "keeps one record per employee, shift and day"

BUG-502 — After the machine's log is cleared, its new punches are dropped without a word — MEDIUM

Area	Attendance, kiosk & machines
Effect	**After the clocking machine's memory is cleared, Sync says "completed" but skips every new punch until the machine's counter passes its old maximum. Those days' attendance is missing from history and payroll.**
What happens	Punches are recognised only by the machine's record number (`userSn`), so every new punch whose number was already seen is taken for a duplicate and skipped, until the count passes the old highest number. Sync still says "Sync completed". Also, a punch without a number is saved as `"undefined"` (`String(undefined)` passes the `!deviceLogId` check), and every later one without a number is dropped.
Steps to reproduce	Sync the machine (punches numbered 1, 2, …). Clear its attendance log (done when its memory fills up); the numbering starts again at 1. Employees clock in; sync again.
Expected	The new punches are saved.
Where	`src/attendance/attendance.service.ts`, `syncLogs` (`deviceLogId = String(log.userSn)`, then `findOne({ deviceLogId })`)
Type	Data loss
Suggested fix	Recognise a punch by employee + time (`deviceUserId` + `recordTime`), as the ZKTeco receiver does with its line hash; skip (and count) entries without them.
Proven by	`attendance.e2e-spec.ts` › "still saves new punches after the machine's log was cleared", "saves punches the machine sent without a record number"

BUG-504 — After a normal day, the kiosk thinks the employee is still clocked in the next morning — MEDIUM

Area	Attendance, kiosk & machines
Effect	**After a normal day (08:00–17:00), the kiosk refuses the employee's clock-in the next morning ("already clocked in") and shows them as clocked in all night. To clock in they must first press Clock out, which records a false punch and a false clock-out.**
What happens	409 "Ali Ben is already clocked in". At 08:00 the 16-hour window starts at 16:00 the day before: it holds the 17:00 clock-out but no longer the 08:00 clock-in, so one punch is counted, which reads as "clocked in". From 00:00 (16 h after 08:00) to 09:00 (16 h after 17:00) the kiosk lists the employee as clocked in since 17:00. Anyone whose rest between two shifts is under 16 hours meets it; to clock in, they have to press Clock out first, which saves a false punch and a false clock-out.
Steps to reproduce	Clock in at 08:00 and out at 17:00 on the kiosk. Next morning at 08:00, clock in again.
Expected	201, clocked in.
Where	`src/attendance/kiosk.controller.ts`, `recentPunches` + `stateOf` (punches of the last 16 hours, odd count = clocked in)
Type	Bug
Suggested fix	Decide from the last punch, not the count: clocked in if the most recent punch within the window is a clock-in. That needs the direction stored on each punch (kiosk punches know it; machine punches have a status code), or counting from the first punch of the employee's current session instead of a fixed window.
Proven by	`kiosk.e2e-spec.ts` › "lets someone who worked 08:00–17:00 yesterday clock in at 08:00 today", "shows someone who clocked out at 17:00 as not clocked in after midnight"

BUG-505 — Overtime after midnight is lost unless the shift ends at exactly 00:00 — MEDIUM

Area	Attendance, kiosk & machines
Effect	**Overtime worked past midnight after an evening shift (for example 14:00–22:00, leaving at 01:00) is recorded as 0, so the employee isn't paid for it.**
What happens	Worked 11:00 (right) but overtime 0: the clock-out (60) is compared with the shift end (1320) without moving it past midnight. Only a shift ending at 00:00 gets that treatment. Worked hours already handle midnight (`if (worked < 0) worked += 1440`).
Steps to reproduce	Shift 14:00–22:00. The employee clocks in at 14:00 and out at 01:00 the next night.
Expected	Worked 11:00, overtime 180 minutes.
Where	`src/attendance/kiosk.controller.ts`, `saveWorktime` (`if (end === 1440 && out < 720) out += 1440`)
Type	Bug (pay)
Suggested fix	Move the clock-out past midnight whenever it is earlier than the clock-in (as for worked hours), and the shift end whenever it is earlier than the shift start; then compare.
Proven by	`kiosk.e2e-spec.ts` › "counts overtime past midnight after an evening shift ending at 22:00"

Low severity · details

18 bugs, sorted by ID.

BUG-001	A MongoDB operator nested more than 20 levels deep gets past the injection check	LOW
Area	Common, tenancy & activity log	
Effect	**No route reads data that deep today, so nothing can be abused now. But the injection check fails open: a future route that reads a deeply nested field would be unprotected.**	
What happens	The request goes through: `hasForbiddenKey` stops looking after 20 levels and answers `false` ("nothing found"), so the deep part is never checked. Today no route reads a field that deep, so the risk is low, but the check fails open.	
Steps to reproduce	Send a body like `{"a":{"a":{ … 25 levels … {"$ne":""}}}}`.	
Expected	400 "The data sent was not valid", like the same operator at a shallow level.	
Where	`src/common/security.ts`, `hasForbiddenKey` (used by `rejectInjection`)	
Type	Security	
Suggested fix	Fail closed: `if (depth > 20) return true;` (a real form never nests that deep).	
Proven by	`security.spec.ts` › "finds an operator hidden deeper than 20 levels"	

BUG-002	Two error messages reach French and Arabic users in English	LOW
Area	Common, tenancy & activity log	
Effect	**French and Arabic users see two error messages in English (a duplicate advance, a work time record that no longer exists) and may not understand them.**	
What happens	The English text. They are built with a variable part, and `PATTERNS` has no entry for them (every fixed message and every other variable message is translated).	
Steps to reproduce	With the app in French or Arabic, give an employee a second advance on the same day, or edit a worktime that was deleted meanwhile.	
Expected	The message in the user's language, like every other API error.	
Where	`src/advances/advances.service.ts` (`An advance for employee … on … already exists.`), `src/worktime/worktime.service.ts` (`Worktime with id … not found`); no matching pattern in `src/common/messages.i18n.ts`	
Type	Bug (translation)	
Suggested fix	Add two patterns, e.g. `/^An advance for employee (.+) on (.+) already exists.?$/` and `/^Worktime with id .+ not found$/`. The worktime one shouldn't show the database id at all: "This work time record no longer exists".	
Proven by	`messages.i18n.spec.ts` › "translates "An advance for employee 12 on 2026-10-20 already exists."" / "translates "Worktime with id … not found""	

BUG-003	A clock-out at exactly 00:00 is logged as "Clock-out time: Changed"	LOW
Area	Common, tenancy & activity log	
Effect	**In the activity log, a clock-out at exactly midnight reads "Clock-out time: Changed" with no time, so an admin checking a night shift can't see when the employee left.**	
What happens	"Clock-out time: Changed", with no time. `"00:00"` is how the backend used to store "not clocked yet", so the log hides it. The frontend now counts a 00:00 punch as a real time (frontend BUG-103) and sends `""` for "no time".	
Steps to reproduce	Night shift 16:00–00:00. The employee clocks out at exactly 00:00 (the page saves `clock_out: "00:00"` with `X-Audit-As: clock`).	
Expected	Activity log: "Clocked out at 00:00".	
Where	`src/audit/audit-changes.ts`, `isBlank` and `clockTime` (both treat `"00:00"` as "no time")	
Type	Bug	
Suggested fix	Treat only `""`/`null` as "no time" in `isBlank` for clock fields and in `clockTime`. Checked in Phase 3: the worktime schema still defaults `clock_in`/`clock_out` to `"00:00"`, and the frontend's clock-in page still sends "no time" as `"00:00"` (`ClockInPage.tsx`, `entryToPayload`). So a real midnight time can't be told from "not clocked" anywhere; fixing it means storing `""`/`null` for "no time" in the schema, the clock-in page and the reports together.	
Proven by	`audit-changes.spec.ts` › "logs a clock-out at exactly 00:00 (end of a night shift) as a clock-out"	

BUG-004	The activity and error logs filter days in UTC, so in Algeria 00:00–00:59 lands on the day before	LOW
Area	Common, tenancy & activity log	
Effect	**When an admin filters the activity or error log by day, changes made between 00:00 and 00:59 appear under the day before, which confuses anyone investigating a night-time change.**	
What happens	It is listed under 19 October, and 21 October 00:00–00:59 shows up under the 20th. The filter uses UTC days; the company works in UTC+1 (`settings.utcOffset`).	
Steps to reproduce	Something is changed at 00:30 on 20 October in Algeria (19 Oct 23:30 UTC). Open the activity log for 20 October.	
Expected	The change is listed under 20 October.	
Where	`src/audit/audit.service.ts` `find` and `src/error-logs/error-log.service.ts` `find` (`new Date(`${query.from}T00:00:00.000Z`)`)	
Type	Bug (time zone)	
Suggested fix	Build the range from the company's `utcOffset` (`2026-10-20T00:00:00+01:00`), as `audit-changes.ts` already does to show dates. The super admin's error log without a company can stay in UTC.	
Proven by	`logs.e2e-spec.ts` › "filters days in the company's time zone"	

BUG-005	A change the database refuses is logged as a 500, with Mongoose's raw text	LOW
Area	Common, tenancy & activity log	
Effect	**Simple refusals (a missing name, an employee number already used) appear in the activity log as server crashes (500) with database jargon, so admins think the system broke.**	
What happens	Status 500 and "Employee validation failed: lastName: Path `lastName` is required., ..." or "E11000 duplicate key error ...". The interceptor sees the database error before `AllExceptionsFilter` turns it into a 400/409, so admins reading the log think the server crashed.	
Steps to reproduce	Add an employee without a name (400 for the user), or with an employee number already used (409).	
Expected	The activity log shows the failed action with 400 / 409 and the message the user saw ("First name is required", "The employee number "6" is already used").	
Where	`src/audit/audit.interceptor.ts`, error handler (`err instanceof HttpException ? err.getStatus() : 500`)	
Type	Bug	
Suggested fix	Share the translation: move `AllExceptionsFilter.translate` into a function both use, and log its status and message.	
Proven by	`logs.e2e-spec.ts` › "logs the refused employee ... with the status the user saw (400)" / "(409)"	

BUG-101	Chrome, Firefox and Edge on phones are listed as another browser	LOW
Area	Sign-in, users & companies	
Effect	**On the "your sessions" list, a phone sign-in from Chrome, Firefox or Edge shows the wrong browser. Someone checking for a sign-in they don't recognise may raise a false alarm or miss a real one.**	
What happens	"Safari on iOS", "Safari on iOS" and "Chrome on Android". On iPhone these browsers say `CriOS/` and `FxiOS/` instead of `Chrome/` and `Firefox/`, and Edge on Android says `EdgA/` instead of `Edg/`. Someone checking for a sign-in they don't recognise sees the wrong browser.	
Steps to reproduce	Sign in from Chrome or Firefox on an iPhone, or from Edge on Android, then open the list of login sessions.	
Expected	"Chrome on iOS", "Firefox on iOS", "Edge on Android".	
Where	`src/auth/device.ts`, `describeDevice`	
Type	Bug	
Suggested fix	Match `/Edg(A|iOS)?//`, `/Chrome/|CriOS//` and `/Firefox/|FxiOS//`.	
Proven by	`device.spec.ts` › "... CriOS ... → Chrome on iOS", "... FxiOS ... → Firefox on iOS", "... EdgA ... → Edge on Android"	

BUG-102	Two admins demoting each other at the same moment leave the company with no admin	LOW
Area	Sign-in, users & companies	
Effect	**If two admins demote or deactivate each other at the same moment, the company has no admin left and needs the super admin to recover.**	
What happens	Both succeed: each request counts the *other* admin as still active before either is saved. The company has no admin left, and only the super admin can fix it. Deactivating or deleting at the same moment has the same gap.	
Steps to reproduce	A company has exactly two admins, A and B. At the same moment A changes B to manager and B changes A to manager (two `PATCH /users/:id`).	
Expected	One of the two is refused ("The company must keep at least one active admin").	
Where	`src/auth/users.service.ts`, `update` → `ensureAnotherActiveAdmin`	
Type	Bug (race condition)	
Suggested fix	Make the check and the change one step: after saving, count active admins and undo if there are none, or run both in a MongoDB transaction.	
Proven by	`users.e2e-spec.ts` › "keeps one active admin when two admins demote each other at once"	

BUG-103	A second company whose name gives the same sign-in link name can't be created	LOW
Area	Sign-in, users & companies	
Effect	**The super admin can't create two companies whose names give the same sign-in link name, and the message doesn't say what to change.**	
What happens	409 "That sign-in link name or one of the device serials is already used by another company". The super admin typed neither, so the message doesn't say what to change.	
Steps to reproduce	Super admin creates "Pizza Roma", then "Pizza Roma!" (or a second "Pizza Roma" in another city), without typing a sign-in link name.	
Expected	The second company is created with its own link name (e.g. `pizza-roma-2`).	
Where	`src/companies/companies.service.ts`, `apply` (`company.slug = slugify(company.name)`) and `save`	
Type	Bug	
Suggested fix	When the link name comes from the company name, add `-2`, `-3`… until it is free. Keep the 409 for a link name the super admin typed.	
Proven by	`companies.e2e-spec.ts` › "creates a second company with the same name"	

BUG-104	A weekly advance share of 0% is refused, with a message saying 0 is allowed	LOW
Area	Sign-in, users & companies	
Effect	**A company can't switch weekly advances off by setting the share to 0%, although the message says 0 is allowed.**	
What happens	400 "Weekly advance ratio must be between 0 and 1": the message says 0 is allowed, but the check refuses it. Hours per day and working days per month use the same wording, though 0 really makes no sense there.	
Steps to reproduce	Company settings → weekly advance share 0% (a company that doesn't give weekly advances): `PATCH /company` with `{ settings: { weeklyAdvanceRatio: 0 } }`.	
Expected	Saved. The frontend accepts 0 (its BUG-002 fix "accept a weekly advance ratio of 0").	
Where	`src/companies/companies.service.ts`, `validateSettings` (`positive(…)` refuses `n <= 0`)	
Type	Bug	
Suggested fix	Allow 0 for `weeklyAdvanceRatio` (refuse only `n < 0`); for the other two, say "must be more than 0 and at most …".	
Proven by	`companies.e2e-spec.ts` › "accepts a weekly advance share of 0 (no weekly advances)"	

BUG-201	An edit can empty an employee's name	LOW
Area	Employees, shifts & tasks	
Effect	**An edit can save an employee with an empty name. Payslips, plannings and reports then show a blank person.**	
What happens	Saved with an empty first name. Required fields and types are only checked when a record is created.	
Steps to reproduce	`PUT /employees/:id` with `{ "firstName": "" }`.	
Expected	400 "First name is required", as when adding an employee.	
Where	`src/employees/employees.service.ts`, `update` (`findByIdAndUpdate` without `runValidators`); same in shifts and tasks	
Type	Bug (validation)	
Suggested fix	Pass `{ new: true, runValidators: true }` to `findByIdAndUpdate` in the employees, shifts and tasks services (worktime already does).	
Proven by	`staff.e2e-spec.ts` › "refuses an edit that empties the employee's name"	

BUG-202 An unknown id answers 200 with an empty body instead of 404 LOW

Area	Employees, shifts & tasks
Effect	**Saving or deleting a record that no longer exists (deleted in another tab) looks successful, so the user believes a change was saved when nothing happened.**
What happens	200 with an empty body. The frontend reports a save as successful when nothing was saved.
Steps to reproduce	`GET`, `PUT` or `DELETE /employees/<id that doesn't exist>` (or an employee that another tab just deleted); same for tasks, and `GET`/`PUT` for shifts.
Expected	404 "... not found", as shifts already do for archive/restore/delete, and advances and bonuses.
Where	`findOne` / `update` / `remove` in `src/employees/employees.service.ts`, `src/shifts/shifts.service.ts` (`findOne`, `update`), `src/tasks/tasks.service.ts`
Type	Bug
Suggested fix	Throw `NotFoundException` when the query returns `null`.
Proven by	`staff.e2e-spec.ts` › "get/put/delete /employees/:id with an unknown id → 404" (8 cases)

BUG-205 The API accepts an employee without an employee number LOW

Area	Employees, shifts & tasks
Effect	**An employee can be saved without an employee number. A second one is then refused with a confusing "number already used" message, and that employee can't be matched to clock-ins.**
What happens	Saved. A second employee without a number is then refused with 409 "This employee number is already used", because the unique index counts "no number" as a value.
Steps to reproduce	`POST /employees` without `empNumber`, or with 0 or a negative number.
Expected	400. The app requires a whole number above 0, and advances, bonuses and work time find employees by that number.
Where	`src/employees/employee.shema.ts` (`empNumber` is optional, but unique per company)
Type	Bug (validation)
Suggested fix	`@Prop({ required: true, min: 1 }) empNumber`.
Proven by	`staff.e2e-spec.ts` › "requires an employee number above 0, as the app does"

BUG-301 Copying a month without the months crashes with a 500 LOW

Area	Planning
Effect	**A broken "copy month" request is logged as a server crash instead of being refused with a clear message.**
What happens	500 "Something went wrong on our side": `undefined.split` throws a TypeError, which is saved to the error log as a crash.
Steps to reproduce	`POST /planning/copy-month` with `{}`, or with a value that isn't a month.
Expected	400 with a plain message ("Choose the month to copy and the month to copy to").
Where	`src/planning/planning.controller.ts` `copyMonth` → `src/planning/planning.service.ts` `copyMonth` (`sourceMonth.split("-")`)
Type	Bug
Suggested fix	Check both values with `/^\d{4}-(0[1-9]|1[0-2])$/` before anything else.
Proven by	`access.e2e-spec.ts` › "POST /planning/copy-month as admin"

BUG-302 Saving a day without the list of entries empties the day, then crashes LOW

Area	Planning
Effect	**A broken "save day" request from another client deletes the whole day's planning, then crashes. The app itself always sends the list, so normal users don't meet it.**
What happens	The day's planning is deleted, then `entries.filter` throws: 500. The app always sends `entries`, so only another client or a broken request hits it.
Steps to reproduce	`POST /planning/bulk` with `{ "planDate": "2026-10-20" }` (no `entries`).
Expected	400, and the day is unchanged.
Where	`src/planning/planning.service.ts`, `replaceDay` (`deleteMany` runs before `entries.filter`)
Type	Bug (crash, data loss)
Suggested fix	Check `Array.isArray(entries)` and a valid `planDate` before deleting anything.
Proven by	`planning.e2e-spec.ts` › "refuses a save without entries, and keeps the day"

BUG-305	"The first Tuesday" of a copied month is the Tuesday saved first, not the earliest one	LOW
Area	Planning	
Effect	**When copying a month, "the first Tuesday" (or other weekday) can be taken from the wrong week, so the copied planning isn't the one the manager expected.**	
What happens	They get Tuesday the 8th, because that was saved first and the source planning isn't sorted by date.	
Steps to reproduce	In September, save Tuesday the 8th, then Tuesday the 1st. Copy September → November.	
Expected	November's Tuesdays get the planning of Tuesday 1 September (the code comments say "the first occurrence").	
Where	`src/planning/planning.service.ts`, `copyMonth` (source planning read without sorting)	
Type	Bug	
Suggested fix	`.sort({ planDate: 1 })` on the source query.	
Proven by	`planning.e2e-spec.ts` › "uses the earliest day of each weekday, whatever order the days were saved in"	

BUG-307	A sub-task without a start time makes the exports crash	LOW
Area	Planning	
Effect	**One sub-task without a start time makes the planning exports fail for that period.**	
What happens	500: `Cannot read properties of undefined (reading 'substring')`. Sub-task times are optional in the schema.	
Steps to reproduce	Plan someone with a sub-task that has a label but no times, then export that day.	
Expected	The export uses the shift's start time.	
Where	`src/planning/planning.service.ts`, the three grid builders (`item.tasks[0].startTime.substring(0, 2)`)	
Type	Bug (crash)	
Suggested fix	Use `item.tasks?.[0]?.startTime || item.shiftId.startTime`.	
Proven by	`planning.e2e-spec.ts` › "exports a day whose first sub-task has no time"	

BUG-402	Advances and bonuses are accepted for an employee who doesn't exist	LOW
Area	Work time, advances & bonuses	
Effect	**An advance or bonus can be recorded for an employee number that doesn't exist. The money appears on no payslip, and later lands on whoever gets that number.**	
What happens	Saved. The money appears in no payslip, and appears on the employee who later gets number 999.	
Steps to reproduce	`POST /advances` or `POST /bonuses` with `emp_id: 999` (no such employee number).	
Expected	400 "Employee not found".	
Where	`src/advances/advances.service.ts` `create`, `src/bonuses/bonuses.service.ts` `create`	
Type	Bug (validation)	
Suggested fix	Check `employeeModel.exists({ empNumber: emp_id })` (company-scoped by the tenant plugin) before saving.	
Proven by	`payroll.e2e-spec.ts` › "refuses an advance or a bonus for an employee who does not exist"	

BUG-503	Clock-in History cuts days at UTC midnight, so in Algeria 00:00–00:59 lands on the day before	LOW
Area	Attendance, kiosk & machines	
Effect	**On Clock-in History, punches between 00:00 and 00:59 appear on the day before, so night-shift arrivals and departures look like they happened the previous day.**	
What happens	It is listed under 19 October. The range is built in UTC; the company works in UTC+1 (`settings.utcOffset`). Night shifts punch in that hour.	
Steps to reproduce	Someone clocks in at 00:30 on 20 October in Algeria (19 Oct 23:30 UTC). Open Clock-in History for 20 October (`GET /attendance/device-logs?from=2026-10-20&to=2026-10-20`).	
Expected	The punch is listed under 20 October.	
Where	`src/attendance/attendance.service.ts`, `getLogsByDateRange` (`new Date(`${from}T00:00:00.000Z`)`)	
Type	Bug (time zone)	
Suggested fix	Same as BUG-004: build the range from the company's offset (`2026-10-20T00:00:00+01:00`).	
Proven by	`attendance.e2e-spec.ts` › "lists a punch at 00:30 in Algeria under that day"	